

Assignment

Homework1

Chen Zhexin

A ARIN5305 Homework Assignment

September 10, 2026

Task 1.2 Report: Can CodeBLEU and an AI Judge Measure Code Semantic Equivalence?

Background and setup

This report asks whether an automatically computed similarity score can replace human judgement of whether two programs do the same thing. Four LeetCode problems were used, each contributing two code pairs: P-LLM, the course's correct solution against the code that DeepSeek-v4 Instant Chat regenerated from my Task 1.1 summary and that passed all LeetCode tests; and P-FLT, the same correct solution against the course's faulty program. CodeBLEU 0.7.0 was run with `lang="java"` and four equally weighted components (n-gram, weighted n-gram, syntax, data flow). The AI judge was DeepSeek-v4 Instant Chat in Instant mode with Deep Think and Search off, one new conversation per pair, using the template below.

```
You are a careful Java code reviewer. Two Java solutions to the same problem are given
below as Solution A and Solution B. Neither one is labelled as correct or incorrect:
judge only from the code itself and do not assume either one is right.
```

```
Please do all of the following:
```

1. In 1-3 sentences each, describe what Solution A and Solution B compute.
2. Decide whether A and B are functionally equivalent, i.e. whether they return exactly the same result for every valid input. Answer YES or NO.
3. If they are not equivalent, give one concrete input where they differ, together with the exact output of A and of B on that input, and explain why the behaviours differ.
4. List the 2-4 most important semantic differences (algorithm, data structures, boundary or edge-case handling, complexity). Ignore cosmetic differences such as variable names, comments, formatting or behaviour-preserving rewrites.
5. Give a semantic equivalence score between 0.00 and 1.00 (1.00 = identical behaviour on every valid input, 0.00 = completely different behaviour) and your confidence in this judgement (high / medium / low).

```
Do not run or simulate the code; reason about it directly.
```

```
Answer using exactly this structure:
```

```
- Behaviour of A:
- Behaviour of B:
- Equivalent: YES / NO
- Semantic equivalence score (0.00-1.00):
- Confidence: high / medium / low
- Key semantic differences:
- Counterexample (if not equivalent):
```

```
Solution A (Java):
```

```
{SOLUTION_A_JAVA}
```

```
Solution B (Java):
```

```
{SOLUTION_B_JAVA}
```

Question (1): CodeBLEU scores

On the same eight pairs, the CodeBLEU scores fall into two groups that do not overlap. The four P-FLT pairs score 0.592 to 0.978, while the four P-LLM pairs reach only 0.270 to

0.378; comparing each reference program with itself returns 1.0000. The high P-FLT scores follow from how those programs were made: each faulty program changes one line or one branch of the reference, so the tokens, the syntax tree and the data-flow graph stay almost identical. The faulty version of `shortest-matching-substring` changes a single character in a boundary condition and still scores 0.9780, including a full-mark data-flow component. The low P-LLM scores come from the opposite case, where the LLM rewrote the implementation: `shortest-matching-substring` and `minimum-swaps-to-sort-by-digit-sum` score only 0.3176 and 0.3783. This reverses my expectation. The components also differ: syntax favours structurally similar code (0.83 to 0.97 across the faulty pairs), n-gram drops most when data structures or variable names change (0.0938), and data flow cannot see a one-character boundary change, so it can read 1.0000 for a program that behaves differently.

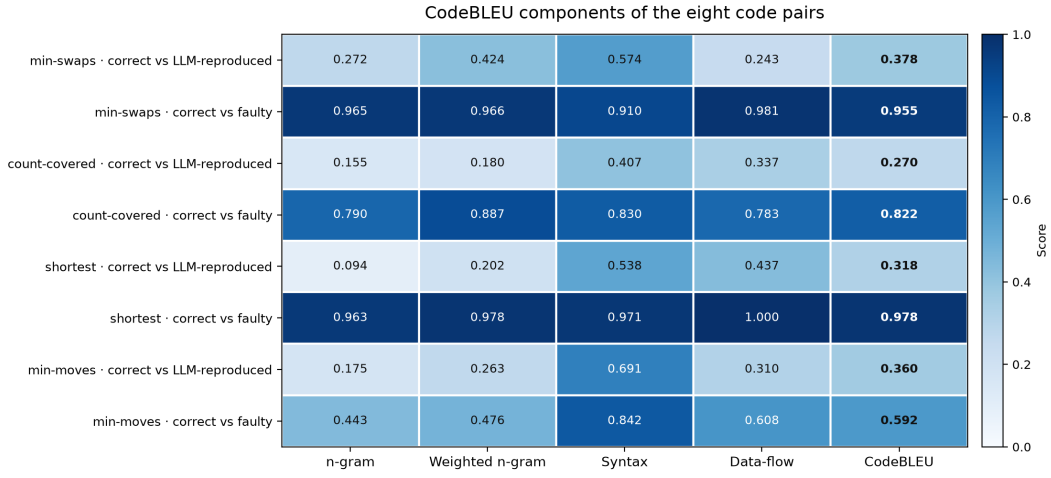


Figure 1: The four CodeBLEU components plus the total, for all eight code pairs. The syntax and data-flow components of the faulty pairs sit above 0.78 and row 6 reaches 1.000 on data flow, while the n-gram component of the LLM-reproduced code falls to 0.094.

Question (2): Key judgements of the AI judge

Unlike a single CodeBLEU number, the AI judge returns a structured answer: what each program computes, a YES or NO verdict, a semantic score between 0 and 1, a confidence level, and a counterexample input where relevant. It gave two YES and six NO verdicts. Both YES answers scored 1.00 and both are P-LLM pairs, on the grounds that the two programs are written differently but compute the same result; all four P-FLT pairs were judged NO with a note on where the program was changed. Seven verdicts were marked high confidence, and only the P-LLM pair of `shortest-matching-substring` was medium. Six answers supplied a counterexample input, recorded here as given (for example `nums=[12,21,30]`, `s="aa"` with `p="a*a*a"`, and `classroom={"S.RL"}` with `energy=1`); I did not treat them as established. The judge is also inconsistent. For the P-LLM pair of `minimum-moves-to-clean-the-classroom`, the same reply first states “Equivalent: NO, score 0.20, confidence high” and later changes to “YES, 1.00, high” (Figure 3). Its semantic scale is not calibrated either: four faulty programs that each change one place receive 0.20,

0.20, 0.75 and 0.85, and one pair is judged NO while scoring 0.85.

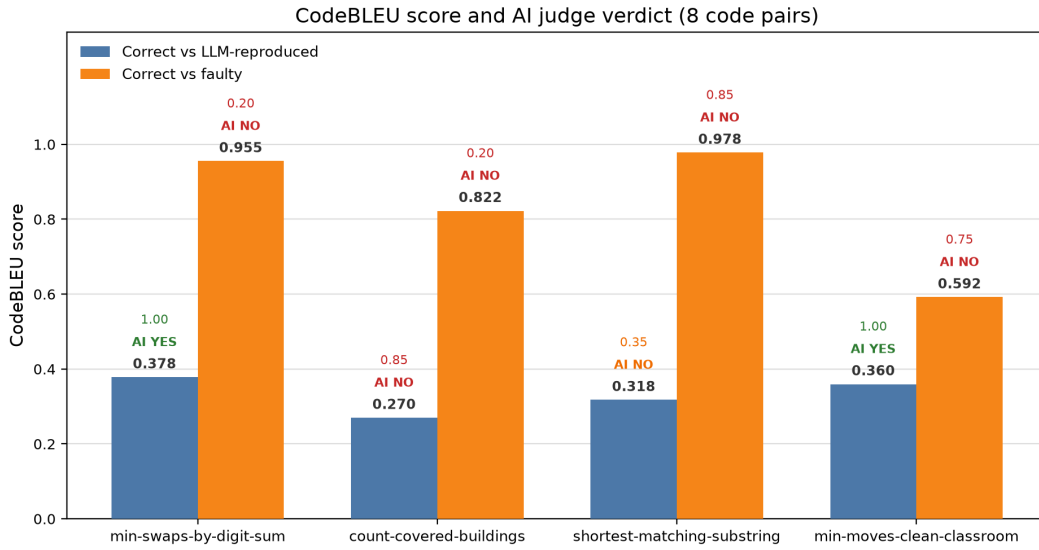


Figure 2: Bar height is the CodeBLEU score; the two labels above each bar give the AI judge verdict and its 0 to 1 semantic score. Blue is the LLM-reproduced pair, orange the faulty pair.

Wait, I need to double-check the pruning in B. The condition is

```
if (best[nx][ny][newMask] >= newEnergy) continue;
```

This means if we have seen this state with energy $\geq$ newEnergy, we skip. But what if we have seen it with energy $<$ newEnergy? Then we update and add to queue. This is correct. However, there is a subtlety: BFS explores level by level. The first time we reach a state, it's with the minimum steps. If we later reach the same state with more steps but higher energy, we might update `best` and add to queue. But since we are doing BFS, the later state has more steps, so even if it has higher energy, it might not lead to a shorter overall path because we already spent more steps to get there. However, the goal is to minimize total steps. If we reach the same state with more steps but higher energy, could that lead to a shorter total path? No, because any continuation from that state will take the same number of

Figure 3: The re-thinking section of DeepSeek’s reply for the LLM-reproduced pair of `minimum-moves-to-clean-the-classroom`, where it revises the judgement it gave earlier in the same reply.

Question (3): Strengths and limitations of the two metrics

CodeBLEU is stable, fast and cheap: repeated runs on the same input agree exactly, the eight pairs take seconds, and no network is needed. It parses syntax trees and data-flow graphs with tree-sitter, so it stays closer to code structure than a plain string comparison (one P-LLM pair has a literal similarity of 0.148 but a CodeBLEU score of 0.3176), and its four components can be inspected separately. That makes it a reasonable bulk pre-filter for plagiarism, clone detection or regression triage. Its score, however, measures similarity of implementation and not sameness of behaviour: a one-character boundary change scores 0.9780 and a min/max rewrite 0.8224, while legitimate rewrites score only 0.3783 and 0.3176.

There is no meaningful cut-off, and a reference program is required. One component is not even strictly deterministic, because variable renaming uses `list(set(...))` and depends on the hash seed; two of the eight pairs vary, by at most about 0.015. The AI judge is the opposite: it reads intent rather than surface form, needs no reference program and explains its reasoning, so it handles rewrites and boundary faults that CodeBLEU misses. Its weaknesses are reliability and auditability: answers are not reproducible, one reply can contradict itself, scores are not comparable across pairs, and every counterexample has to be confirmed.

In summary, I do not think either method should be used as the single basis for a judgement. CodeBLEU can serve as a cheap, reproducible pre-filter, while the AI judge can supply reasons but needs verification: every difference it claims should be confirmed before being cited, so that hallucination problems are avoided.